SUBRUTINAS

El diseño de un programa que resuelve un determinado problema puede simplificarse si se plantea adecuadamente la utilización de subrutinas. Éstas permiten dividir un problema largo y complejo en subproblemas más sencillos o módulos, más fáciles de escribir, depurar y probar que si se aborda directamente el programa completo. De esta forma se puede comprobar el funcionamiento individual de cada rutina y, a continuación, integrar todas en el programa que constituye el problema global de partida. Otra ventaja que aporta la utilización de subrutinas es que en ocasiones una tarea aparece varias veces en el mismo programa; si se utilizan subrutinas, en lugar de repetir el código que implementa esa tarea en los diferentes puntos, bastará con incluirlo en una subrutina que será invocada en el programa tantas veces como sea requerida. Yendo más lejos, subproblemas de uso frecuente pueden ser implementados como rutinas de utilidad (librerías), que podrán ser invocadas desde diversos módulos.

Con el fin de dotar de generalidad a una subrutina, ésta ha de ser capaz de resolver un problema ante diferentes datos que se le proporcionan como parámetros de entrada cuando se la llama. Por tanto, para una correcta y eficaz utilización de las subrutinas es necesario tener en cuenta, por un lado, la forma en que se realizan las *llamadas* y, por otro, el *paso de parámetros*.

En esta práctica se comenzará con la presentación de algunos ejercicios sencillos que permitan la familiarización con el manejo de la pila, estructura imprescindible para una buena gestión de las subrutinas. Posteriormente se introducirán las técnicas de llamada y paso de parámetros.

1. Gestión de la pila

Una pila o cola LIFO (Last Input First Output) es una estructura de datos caracterizada por que el último dato que se almacena es el primero que se obtiene después. Para gestionar la pila se necesita un puntero a la última posición ocupada de la misma, con el fin de conocer dónde se tiene que dejar el siguiente dato a almacenar, o para saber dónde están situados los últimos datos almacenados en ella. Para evitar problemas, el puntero de pila siempre debe estar apuntando a una palabra de memoria. Por precedentes históricos, el segmento de pila siempre "crece" de direcciones superiores a direcciones inferiores. Las dos operaciones más típicas con esta estructura son:

- Transferir datos hacia la pila (*push* o apilar). Antes de añadir un dato a la pila se tienen que restar 4 unidades al puntero de pila.

- Transferir datos desde la pila (*pop* o desapilar). Para eliminar datos de la pila, después de extraído el dato se tienen que sumar 4 unidades al puntero de pila.

El ensamblador del MIPS tiene reservado un registro, $sp, como puntero de pila (stack pointer). Para realizar una buena gestión de la pila será necesario que este puntero sea actualizado correctamente cada vez que se realiza una operación sobre la misma.

El siguiente fragmento de código muestra cómo se puede realizar el apilado de los registros $t0 y $t1 en la pila:

```
            .text
main:       li $t0,10
            li $t1, 13     #inicializar reg. t0,$t1
            addi $sp, $sp, -4  #actualizar el sp
            sw   $t0, 0($sp)   #apilar t0
            addi $sp, $sp, -4  #actualizar el sp
            sw   $t1, 0($sp)   #apilar t1
```

Edita el programa, reinicializa el simulador y carga el programa.

Cuestión 1: Ejecuta el programa paso a paso y comprueba en qué posiciones de memoria, pertenecientes al segmento de pila se almacena el contenido de los registros $t0 y $t1.

Cuestión 2: Modifica el programa anterior para que en lugar de actualizar el puntero de pila cada vez que se pretende apilar un registro en la misma, se realice una sola vez al principio y después se apilen los registros en el mismo orden.

Cuestión 3: Añade el siguiente código al programa original después de la última instrucción:

- Modifica el contenido de los registros $t0 y $t1, realizando algunas operaciones sobre ellos.

- Recupera el contenido inicial de estos registros desapilándolos de la pila, y actualiza el puntero de pila correctamente (limpiar la pila).

Ejecuta el programa resultante y comprueba si finalmente $t0 y $t1 contienen los datos iniciales que se habían cargado. Comprueba también que el contenido del registro $sp es el mismo antes y después de la ejecución del programa.

Cuestión 4: Implementa el siguiente programa:

- Almacena en memoria una tira de caracteres de máximo 10 elementos.

- Lee la cadena de caracteres desde el teclado.

- Invierte esta cadena, almacenando la cadena resultante en las mismas posiciones de memoria que la original. (Sugerencia para realizar el ejercicio: Se apilan los elementos de la tira en la pila y, a continuación, se desapilan y se almacenan en el mismo orden que se extraen).

2. Llamada y retorno de una subrutina

El juego de instrucciones del MIPS R2000 dispone de una instrucción específica para realizar la llamada a una subrutina, `jal etiqueta`. La ejecución de esta instrucción conlleva dos acciones:

- Almacenar la dirección de memoria de la siguiente palabra a la que contiene la instrucción `jal` en el registro $ra.

- Llevar el control de flujo de programa a la dirección `etiqueta`.

Por otra parte, el MIPS R2000 dispone de una instrucción que facilita el retorno de una subrutina. Esta instrucción es `jr $ra`, instrucción de salto incondicional que salta a la dirección almacenada en el registro $ra, que justamente es el registro dónde la instrucción `jal` ha almacenado la dirección de retorno cuando se ha hecho el salto a la subrutina.

Para ejecutar cualquier programa de usuario el simulador XSPIM hace una llamada a la rutina `main` mediante la instrucción `jal main`. Esta instrucción forma parte del código que añade éste para lanzar a ejecución un programa de usuario. Si la etiqueta `main` no está declarada en el programa se genera un error. Esta etiqueta deberá siempre referenciar la primera instrucción ejecutable de un programa de usuario. Para que cualquier programa de usuario termine de ejecutarse correctamente, la ultima instrucción ejecutada en éste debe ser `jr $ra` (salto a la dirección almacenada en el registro $ra), que devuelve el control a la siguiente instrucción desde donde se lanzó la ejecución del programa de usuario. A partir de este punto se hace una llamada a una función del sistema que termina la ejecución correctamente.

El siguiente código es un programa que realiza la suma de dos datos contenidos en los registros $a0 y $a1.

```
          .text
main:     li $a0,10
          li $a1,20
          add $v1,$a0,$a1
          jr $ra
```

Reinicializa el simulador, carga el programa y ejecútalo paso a paso, contestando a las siguientes cuestiones:

Cuestión 5: ¿Cuál es el contenido del PC y del registro $ra antes y después de ejecutar la instrucción `jal main`?

Cuestión 6: ¿Cuál es el contenido de los registros PC y $ra antes y después de ejecutar la instrucción `jr $ra`?

Cuestión 7: Reinicializa el simulador, carga el programa de nuevo y ejecútalo todo completo. Comprueba que la ejecución termina correctamente sin que salga el mensaje de error que salía en ejecuciones anteriores cuando no se incluía la instrucción `jr $ra`.

3. Anidado de subrutinas

El anidado de subrutinas se produce cuando se encadenan dos o más llamadas a subrutinas desde otras subrutinas. Esta circunstancia es muy habitual cuando se trabaja con librerías y se están diseñando programas complejos y bien estructurados.

Cuando una subrutina llama a otra utilizando la instrucción `jal` se modifica automáticamente el contenido del registro $ra con la dirección de retorno (dirección de memoria de la siguiente palabra a la que contiene la instrucción que ha efectuado el salto, es decir, la dirección de la instrucción posterior al salto). Esto hace que se pierda cualquier contenido anterior que pudiera tener este registro, que podría ser a su vez otra dirección de retorno suponiendo que se llevan efectuadas varias llamadas anidadas. Así pues, en cada llamada a una subrutina se modifica el contenido del registro $ra, y sólo se mantiene en este registro la dirección de retorno asociada a la última llamada, ¿Qué ocurre entonces con todas las direcciones de retorno que deben guardarse en las llamadas anidadas?

Una de las soluciones a este problema es que antes de ejecutar una llamada desde una subrutina a otra se salve el contenido del registro $ra en la pila. Como la pila crece dinámicamente, las direcciones de retorno de las distintas llamadas anidadas quedarán almacenadas a medida que éstas se van produciendo. La última dirección de retorno apilada estará en el tope de la pila, y ésta es justamente la primera que se necesita recuperar. Así pues, una pila es la estructura adecuada para almacenar las direcciones de retorno en llamadas anidadas a subrutinas.

El siguiente código implementa una llamada anidada a dos subrutinas, rutina `main` y `subr`. A la primera se le llamará desde el fragmento de código que tiene el simulador para lanzar la ejecución de un programa de usuario y a la subrutina `subr` se la llamará desde `main`. La primera necesita apilar la dirección de retorno (contenido de $ra) para que sea posible la vuelta a la instrucción siguiente desde donde se hizo la llamada a la rutina `main`. La segunda como no hace ninguna llamada a otra subrutina no necesita apilar el contenido de $ra.

La descripción algorítmica se muestra a continuación:

```
PROGRAMA EJEMPLO-1
VARIABLES
     ENTERO: dato1=10; dato2=20; suma;
INICIO
     suma=subr(dato1,dato2);
FIN

FUNCION subr(ENTERO: para1, para2):ENTERO;
INICIO
     subr=para1+para2;
FIN
```

Y el programa en ensamblador quedaría como sigue:

```
          .data
suma:     .space    4
dato1:    .word     10
dato2     .word     20
          .text
main:     addi $sp,$sp,-4    #apilar dir. ret.
          sw   $ra,0($sp)
          lw   $a0,dato1($0)
          lw   $a1,dato2($0)
          jal  subr
          sw   $v0,suma($0)
          lw   $ra,0($sp)    #desapilar dir. ret.
          addi $sp,$sp,4
          jr   $ra
subr:     add  $v0,$a0,$a1
          jr   $ra
```

Reinicializa el simulador, carga el programa y ejecútalo paso a paso respondiendo a las siguientes cuestiones:

Cuestión 8: Comprueba qué contiene el registro $ra y el PC antes y después de ejecutar la instrucción `jal main`.

Cuestión 9: Comprueba qué hay almacenado en el tope de la pila después de ejecutar las dos primeras instrucciones del programa .

Cuestión 10: Comprueba el contenido de los registros $ra y PC antes y después de ejecutar la instrucción `jal subr`.

Cuestión 11: Comprueba el contenido de los registros $ra y PC antes y después de ejecutar la instrucción `jr $ra` que está en la subrutina subr.

Cuestión 12: Comprueba el contenido de los registros $ra y PC antes y después de ejecutar la instrucción `jr $ra` que está en la subrutina main. ¿Qué hubiera ocurrido si antes de ejecutar la instrucción `jr $ra` de esta subrutina `main` no se hubiese desapilado el contenido del registro $ra?

Cuestión 13: Añade el código necesario para realizar las siguientes modificaciones sobre el código original:

La subrutina `subr` debería calcular la siguiente operación dato1/dato2. Esta división sólo se realizará si ambos operandos son mayores que 0. La comprobación se realizará en una segunda subrutina llamada `comp` que devolverá un 1, si ambos datos, `dato1` y `dato2` son mayores que 0, y un 0 en caso contrario. La descripción algorítmica del programa propuesto se muestra a continuación:

```
PROGRAMA EJEMPLO-2
VARIABLES
     ENTERO: dato1=10; dato2=20; division;
INICIO
     division=subr(dato1, dato2);
FIN

FUNCION comp(ENTERO: para1, para2):ENTERO;
INICIO
     Si ((para1>0) and (para2>0)) ENTONCES
          comp=1;
     Sino
          comp=0;
     FinSi
FIN
```

```
FUNCION subr(ENTERO: para1, para2):ENTERO;
VARIABLES
     ENTERO: mayor;
INICIO
     Si (comp(para1,para2)=1)
          subr=para1/para2;
     Sino
          subr=-1;
     FinSi
FIN
```

4. Paso de parámetros

A la hora de implementar una subrutina hay que decidir:

- Parámetros a pasar a la rutina.
- Tipo de parámetros:

  por valor:

  - se pasa el valor del parámetro,

  por referencia:

  - se pasa la dirección de memoria donde está almacenado el parámetro.
- Lugar donde se van a pasar los parámetros:

  - registros,
  - pila

El ensamblador del MIPS establece el siguiente convenio para realizar el paso de parámetros:

- Los 4 primeros parámetros de entrada se pasarán a través de los registros $a0 - $a3. A partir del quinto parámetro se pasaría a través de la pila.
- Los dos primeros parámetros de salida se devuelven a través de los registros $v0 - $v1, el resto a través de la pila.

El siguiente programa implementa la llamada a una subrutina y el código de la misma, que devuelve una variable booleana que vale 1 si una determinada variable está dentro de un rango y 0, en caso contrario. La descripción algorítmica del mismo se muestra a continuación:

```
PROGRAMA EJEMPLO-3

VARIABLES

     ENTERO: rango1=10; rango2=50; dato=12; res;

INICIO

     res=subr(rango1,rango2,dato);

FIN


FUNCION subr(ENTERO: para1, para2, para3):ENTERO;

INICIO

     Si ((para3>=para1)and(para3<=para2)) ENTONCES

          subr=1;

     Sino

          subr=0;

     FinSi

FIN
```

La subrutina tendrá, pues, los siguientes parámetros:

- Parámetros de entrada:
  - Las dos variables que determinan el rango, pasados por valor, a través de $a0 y $a1.
  - La variable que se tiene que estudiar si está dentro del rango, pasada por valor, a través de $a2.
- Parámetros de salida:
  - Variable booleana que indica si la variable estudiada está o no dentro del rango, por valor, devuelto a través de $v0.

El listado del programa en ensamblador se muestra a continuación:

```
          .data
rango1:   .word    10
rango2:   .word    50
dato:     .word    12
res:      .space   1
          .text
main:     addi $sp,$sp,-4
          sw $ra,0($sp)      #apilar ra
          lw $a0,rango1($0)  #a0=rango1
          lw $a1,rango2($0)  #a1=rango2
          lw $a2,dato($0)    #a2=dato
          jal subr      #saltar a subr
          sb $v0,res($0)     #res=v0
          lw $ra,0($sp)
          add $sp,$sp,4 #desapilar ra
          jr $ra        #terminar ejecucion programa
subr:     blt $a2,$a0,sino   #Si a2<a0 saltar a sino
          bgt $a2,$a1,sino   #si a2>a1 saltar a sino
entonces: addi $v0, $0,1     #v0=1
          j finsi            #saltar a finsi
sino:     add $v0,$0,$0      #v0=0
finsi:    jr $ra             #retornar
```

Reinicializa el simulador, carga el programa, ejecútalo y comprueba el resultado almacenado en la posición de memoria `res`. Contesta a las siguientes cuestiones:

Cuestión 14: Identifica las instrucciones que se necesitan en:

- El programa que hace la llamada para:

  a) La carga de parámetros en los registros.

b) La llamada a la subrutina.

c) El almacenamiento del resultado.

- La subrutina para:

a) La lectura y procesamiento de parámetros.

b) La carga del resultado en $v0

c) El retorno al programa que ha hecho la llamada.

Cuestión 15: Modifica el código anterior para que los parámetros que se pasan a la subrutina `subr`, tanto los de entrada como el de salida, se pasen por referencia. La descripción algorítmica del programa se muestra a continuación:

```
PROGRAMA EJEMPLO-4
VARIABLES
     ENTERO: rango1=10; rango2=50; dato=12; res;
INICIO
     subr(rango1,rango2,dato,res);
FIN


PROCEDIMIENTO subr(ENTERO: para1,para2;para3;
                   VAR ENTERO: para4);
INICIO
     Si (para3>=para1)or(para3<=para2)) ENTONCES
          para4=1;
     Sino
          para4=0;
     FinSi
FIN
```

Los pasos a realizar tanto por el programa que hace la llamada como por la subrutina son los siguientes:

- En el programa que hace la llamada:

  a) Cargar la dirección de los parámetros en los registros correspondientes (entrada y salida).

  b) Llamada a la subrutina.

- En la subrutina:

  a) Lectura de los parámetros de entrada a partir de las direcciones pasada como parámetro.

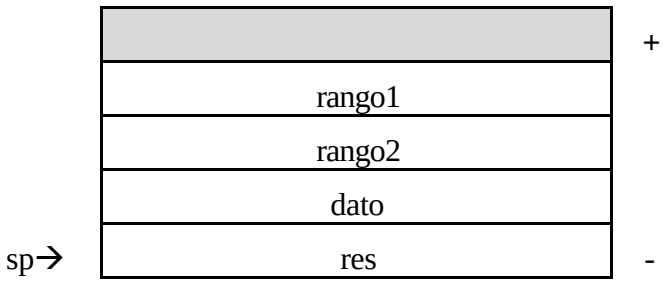
  b) Procesamiento de los parámetros de entrada y generación del resultado.

  c) Almacenamiento del resultado en la dirección de memoria pasada a través del parámetro de salida

  d) Retorno al programa que hizo la llamada.

Ejecuta el programa obtenido y comprueba que el resultado obtenido es el mismo que el del programa original.

Cuestión 16: Modifica el código anterior para que los parámetros de entrada a la subrutina se pasen por valor mediante la pila y el de salida se pase también a través de la pila pero por referencia. A continuación se muestra un esquema de cómo debe ser la situación de la pila en el momento que empieza a ejecutarse la subrutina:



Problemas propuestos

1. Implementa una subrutina en ensamblador que calcule cuántos elementos de un vector de enteros de dimensión n son iguales a un elemento dado:

   - Parámetros de entrada a la subrutina:
     - dirección del primer elemento del vector,
     - total de elementos del vector (dimensión),
     - elemento a comparar.
   - Parámetro de salida de la subrutina:
     - contador calculado.

   Realiza las siguientes implementaciones:

   a) Implementa la subrutina de forma que todos los parámetros que se le pasan sean por valor excepto aquéllos que obligatoriamente se deban pasar por referencia, y a través de registros.

   b) Implementa la subrutina de forma que todos los parámetros se pasen por referencia y a través de registros.

   c) Implementa la subrutina donde los parámetros que se pasan sean del mismo tipo que en el caso a) pero utilizando la pila como lugar para realizar el paso de parámetros.

2. Implementa una subrutina en ensamblador, tal que dado un vector de enteros de dimensión *n* obtenga el elemento (i) de dicho vector. La subrutina tendrá como parámetros de entrada: la dirección del vector, la dimensión del mismo y el índice del elemento a devolver. La subrutina devolverá el elemento i-ésimo. Realiza la llamada y el retorno a la subrutina según el convenio establecido.

3. Implementa una subrutina en ensamblador, tal que dada una matriz de enteros de dimensión n x m, almacenada por filas, obtenga el elemento (i,j) de dicha matriz. La subrutina tendrá como parámetros de entrada: la dirección de la matriz, las dimensiones de la misma y los índices del elemento a devolver. La subrutina devolverá el elemento (i,j). Realiza la llamada y el retorno a la subrutina según el convenio establecido.